AssertionForge: Enhancing Formal Verification Assertion Generation with Structured Representation of Specifications and RTL

Yunsheng Bai, Ghaith Bany Hamad, Syed Suhaib, Haoxing Ren
NVIDIA
yunshengb, gbanyhamad, ssuhaib, haoxingr@nvidia.com

arXiv:2503.19174v2 [cs.AI] 14 May 2025

***Abstract*—Generating SystemVerilog Assertions (SVAs) from natural language specifications remains a major challenge in formal verification (FV) due to the inherent ambiguity and incompleteness of specifications. Existing LLM-based approaches, such as ASSERTLLM, focus on extracting information solely from specification documents, often failing to capture essential internal signal interactions and design details present in the RTL code, leading to incomplete or incorrect assertions. We propose a novel approach that constructs a Knowledge Graph (KG) from both specifications and RTL, using a hardware-specific schema with domain-specific entity and relation types. We create an initial KG from the specification and then systematically fuse it with information extracted from the RTL code, resulting in a unified, comprehensive KG. This combined representation enables a more thorough understanding of the design and allows for a multi-resolution context synthesis process which is designed to extract diverse verification contexts from the KG. Experiments on five designs demonstrate that our method significantly enhances SVA quality over prior methods. This structured representation not only improves FV but also paves the way for future research in tasks like code generation and design understanding.**

***Index Terms*—Formal Verification, Large Language Model, SystemVerilog Assertion, Knowledge Graph**

I. INTRODUCTION

Formal Verification (FV) is a critical process in the VLSI design flow, ensuring that increasingly complex hardware designs conform to their specifications [11]. Among FV techniques, Assertion-Based Verification (ABV) uses SystemVerilog Assertions (SVAs), which are formal properties that describe expected design behavior, to verify functional correctness [58]. To be effective, these assertions must encapsulate both the high-level design intent, as described (often ambiguously) in natural language specifications [19], and the low-level implementation details codified in Register-Transfer Level (RTL) code. Verification engineers traditionally bridge this gap by mentally synthesizing information from both sources, a process that is difficult to automate yet essential for producing high-quality SVAs—assertions that are syntactically correct, functionally accurate, and provide comprehensive coverage of critical design properties.

Recently, Large Language Models (LLMs) have shown promise in generating SVAs from natural language specifications, thanks to their advanced text comprehension and coding capabilities. Methods like ASSERTLLM [18] decompose the SVA generation task into phases, using customized LLMs to extract structural specifications, map signal definitions, and generate assertions by prompting the LLM with contextual information from specifications about relevant signals. These approaches demonstrate the feasibility of using LLMs for generating SVAs, but they remain fundamentally limited in their ability to capture the full design context.

The limitations stem from their approach to utilizing sources of verification information. Specification-focused approaches like ASSERTLLM extract design intent from the often ambiguous and incomplete specifications [3], [16], [19], [21], [52], and thus miss crucial implementation details found exclusively in the RTL, leading to plausible but functionally incorrect assertions. Conversely, RTL-focused approaches can capture implementation details but lack understanding of original design intent and high-level functionality, producing assertions that verify implementation without validating adherence to specifications.

We hypothesize that explicitly constructing a structured, interconnected mental model of the design—one that links design intent with RTL behavior—will provide a more robust foundation for assertion generation. This mirrors how verification engineers manually synthesize information from multiple sources to write meaningful assertions. Compared to unstructured retrieval-based approaches [18], [33]. such a mental model enables more precise tracing of intent to implementation and facilitates deeper reasoning, allowing the LLM to identify signal dependencies and complex interactions. However, building such a model is challenging, requiring structured knowledge extraction from specifications, analysis of RTL control flows, and alignment between abstract requirements and implementation details.

In this paper, we present ASSERTIONFORGE, a novel approach that constructs a unified **Knowledge Graph (KG)** from specifications and RTL, thereby actively "forging" assertions by bridging the semantic gap between high-level specifications and RTL implementation, enabling discovery of complex multi-signal interactions often obscured in linear text. The KG is built in two stages: entity-relation extraction from the specification via an LLM and structural parsing of RTL via a Verilog parser.

With the unified KG constructed, the next step is crafting

effective prompts for LLMs to generate high-quality test plans and SVAs. We propose a **multi-resolution context synthesis** process that progressively narrows focus from broad design overviews to fine-grained signal interactions: (1) LLM-based global summarization provides a high-level view of the specification and RTL; (2) Signal-specific retriever extracts relevant snippets from the specification and RTL; (3) **Guided Random Walk with Adaptive Sampling** (GRW-AS) explores the KG, tracing structured paths that connect the target signal to critical design elements. To ensure concise yet informative prompts, an LLM-based pruning mechanism filters and refines the retrieved text and graph-derived paths, balancing diversity and relevance before guiding final SVA generation.

Our contributions are summarized as follows:

- We propose a novel KG-based approach, ASSERTIONFORGE, that integrates information from both natural language specifications and RTL code into a unified structured representation for SVA generation.
- We develop a multi-resolution context synthesis process, which constructs verification-critical prompts by providing context as global summaries, signal-specific snippets, and structured paths from the KG.
- We demonstrate that our method significantly enhances SVA quality through experiments on five designs, achieving superior results compared to existing LLM-based methods.

II. Related Work

A. Formal Verification and SVA Generation

LLM-based approaches for SystemVerilog Assertion (SVA) generation have gained traction [2], [4], [28], [29], [31], [36]–[38], [44], [47], with ASSERTLLM [18] automating assertion extraction from specifications. Non-LLM methods include GOLDMINE [57], HARM [20], A-Team [15], and AutoSVA [43], which use data mining, static analysis, templates, etc. for assertion generation. However, these approaches often struggle with complex specifications, suffer from limited coverage, or depend heavily on simulation stimuli quality. Recent RTL-guided SVA works include [27] and [54]. Our work focuses on advancing LLM-based SVA generation, exploring how structured representation and context-aware prompting can improve assertion quality.

B. LLM for Knowledge Graph Construction and Understanding

Traditional methods for Knowledge Graph (KG) [1], [25] construction rely on non-LLM approaches requiring manual effort and domain-specific ontologies [9], [39], [62], [63]. Recent advancements leverage LLMs for KG construction, automatically inferring entities and relationships [7], [45], [56], [64]. LLM-based graphs and KGs have gained popularity across domains [30], [46], with Microsoft's GraphRAG demonstrating their application in question answering [17]. KGs enhance LLM-based NLP tasks including question answering, dialogue systems, and reasoning [35], [49], [53], [59], [60] by improving knowledge-grounded responses and contextual understanding. Our approach adapts these advancements to hardware verification by integrating specifications and RTL code into a knowledge-rich graph.

III. Methodology

In this section, we introduce our novel approach, ASSERTIONFORGE (Figure 1), for generating high-quality SystemVerilog Assertions (SVAs) from both natural language specifications and RTL code.

A. Problem Setup and Overview of ASSERTIONFORGE

We formalize the problem as follows: Given a hardware design specification $\mathcal{S}$ and RTL code $\mathcal{R}$, we generate SystemVerilog Assertions $\mathcal{A} = \{A_1, \ldots, A_n\}$ evaluated for syntactic correctness and functional accuracy. To emulate verification engineers' cognitive process [19] — a workflow rooted in cognitive science's concept of **mental model** (internal representations humans construct to understand complex systems [26]) — we develop a computational approach integrating $\mathcal{S}$ (design intent) and $\mathcal{R}$ (implementation details).

To computationally realize this mental model, we construct a Knowledge Graph (KG), $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ represents nodes (entities) and $\mathcal{E}$ represents edges (relationships). Entities $\mathcal{V}$ are derived from both $\mathcal{S}$ and $\mathcal{R}$, and edges $\mathcal{E}$ capture relationships defined by our domain-specific schema. Formally, $\mathcal{G} = \mathcal{F}(\mathcal{S}, \mathcal{R})$, where $\mathcal{F}$ is the KG generation function. In our approach, the complete mental model comprises both this structured KG representation alongside the raw specification and RTL texts, enabling context-aware SVA generation through dynamically constructed LLM prompts.

B. Knowledge Graph Construction with GraphRAG

GraphRAG [17] is a framework for generating knowledge graphs from textual documents through a process of **chunking** source text, **LLM-based extraction** of entities and relationships, **summarization** of node/edge descriptions, and **graph assembly**. While GraphRAG's default extraction focuses on general-purpose entities, hardware verification demands specialized entity and relationship types.

We propose an effective procedure to customize GraphRAG for hardware verification. First, we leverage **automated LLM-based schema discovery** by presenting a small representative set of hardware specification excerpts to the LLM, prompting it to identify domain-specific entity types (e.g., "registers", "clocks") and relation types (e.g., "triggers", "connects_to"). This is followed by **schema review**, where we manually refine the LLM-generated schema by merging redundant or overlapping concepts (e.g., combining "Reset_Signal" and "Reset"). Finally, through **prompt update and indexing**, we modify only GraphRAG's entity extraction prompt to incorporate our hardware-specific schema while keeping all other GraphRAG indexing components unchanged.

The schema Σ is formally defined as a tuple $(\mathcal{E}_t, \mathcal{R}_t)$ consisting of **entity types** $\mathcal{E}_t$ representing key hardware design components (e.g., "design specification", "module", "signal",

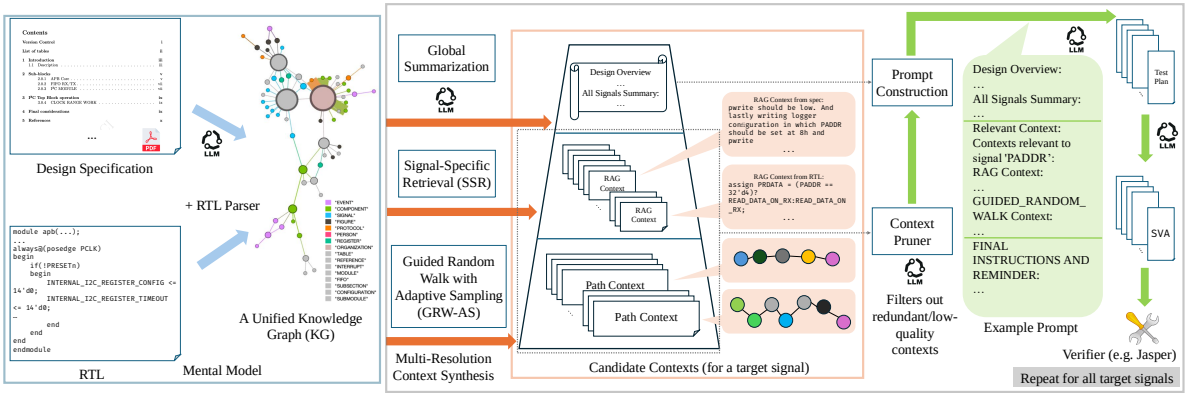

Fig. 1. Overview of ASSERTIONFORGE. Our method is structured into a three-stage process: First, we construct a domain-specific Knowledge Graph (KG) that captures the essential entities and relationships in the design. Together with the raw specifications and RTL, we build a mental model for the design under verification. Second, we generate three types of candidate contexts for each signal. Lastly, we use these contexts to dynamically construct prompts for a Large Language Model (LLM) to generate accurate SVAs.

"port", "register", "clock") and **relation types** $\mathcal{R}_t$ representing relationships (e.g., "hasSection", "contains", "defines", "connectsTo", "operatesAt").

Our customized entity extraction prompt ensures that nodes and edges in $\mathcal{G}_0$ contain detailed attributes such as type, description, source ID, and relationships. Node attributes include fields like entity type, description, level, and degree, while edge attributes capture weight, description, and source IDs of connected nodes. These attributes are crucial for accurately representing both high-level and low-level design aspects. We will publicly release the exact prompt used for entity extraction and our entire source code to enable reproducibility.

C. Refinement with RTL Information

To enhance the completeness and accuracy of the KG, we further refine $\mathcal{G}_0$ using information extracted from the RTL code $\mathcal{R}$. We utilize a specialized Verilog parser based on [55] to extract detailed information from RTL design files, including modules, ports, signals, control flow structures, and signal assignments. This process, denoted as $\mathcal{G} = \psi(\mathcal{G}_0, \mathcal{R})$, analyzes Verilog files to augment the initial KG with RTL-specific elements. New nodes are added for internal signals, control flow constructs, and module instances, while existing nodes are enriched with attributes like signal width and type information. Edges represent both structural relationships (e.g., "has_port") and behavioral relationships (e.g., "assigns_to"). Our analysis also identifies finite state machines by recognizing clocked always blocks with case statements, creating FSM nodes that capture state transition logic.

Correspondences between specification and RTL nodes are identified via string matching of signal and module names. We use exact matching for direct references and a fuzzy matching algorithm to handle minor variations in naming conventions. This linking process establishes traceability between requirements and implementation.

D. Multi-Resolution Context Synthesis for SVA Generation

Current approaches to LLM-based hardware verification predominantly rely on Retrieval-Augmented Generation (RAG) [18], [33]. While RAG can effectively retrieve documentation snippets and code fragments using signal names as queries, the retrieved context often contains noise, and important behavioral patterns may be scattered across multiple documents and not captured in a single retrieval.

To address these challenges, we propose a dynamic context synthesis framework that unifies multiple context generators with an intelligent LLM-based pruning mechanism. Our framework consists of a suite of context generators where each generator leverages distinct heuristics to capture different aspects of the design:

(i) **Global Summarization:** (broadest level) An LLM-based summarizer extracts high-level design intent from specifications and RTL, forming a cohesive system-level overview.

(ii) **Signal-Specific Retrieval (SSR):** Given a target architectural signal, a RAG-based retriever extracts localized, functionally relevant snippets from the specification and RTL, ensuring precise contextual grounding.

(iii) **Guided Random Walk with Adaptive Sampling (GRW-AS):** Given a node in the KG corresponding to a target signal, a novel graph-based traversal algorithm (detailed in Section III-E) explores the KG, generating paths that connect the target signal to other key design elements.

Since naively combining all contexts would overwhelm downstream processing, we introduce an *LLM-as-Pruner*. This pruner evaluates each candidate context generated by SSR and GRW-AS — evaluating relevance, diversity, and verification potential — and filters out redundant or low-quality contexts. The pruner considers both the individual merit of each context and its complementarity with other selected contexts to ensure comprehensive coverage of verification properties. Due to space constraints, we omit the full LLM-pruner prompt; however, it will be released upon paper acceptance.

The pruned contexts are then packed into dynamic prompts—up to the maximum token limit—to provide the LLM with comprehensive yet focused information. This ensures that the LLM does not just process more data, but pro-

cesses better-selected data. By dynamically adapting context selection based on verification needs, our approach enables the LLM to generate test plans and SystemVerilog Assertions (SVAs) that capture complex behavioral patterns that are missed by traditional RAG-only methods.

Following ASSERTLLM, we define architectural signals as input/output ports and architectural-level registers explicitly mentioned in the specification, excluding internal signals further implemented in RTL. For each architectural signal, we construct a dedicated prompt by combining the global summaries with the LLM-pruned contexts from SSR and GRW-AS. These contexts are packed across multiple prompts, each varying in content but constrained by the LLM's maximum token limit, ensuring diverse yet focused verification inputs. Each enriched prompt is then used to generate natural language (NL) test plans and actual SystemVerilog Assertions (SVAs). The generated test plans and SVAs for all architectural signals are combined for final evaluation.

E. Guided Random Walk with Adaptive Sampling (GRW-AS)

To efficiently explore the KG's structural and behavioral relationships, we introduce the Guided Random Walk with Adaptive Sampling (GRW-AS) algorithm. Inspired by biased random walk techniques [61] used in recommendation systems [12], knowledge discovery [5], network analysis [13], and graph representation learning [22], [23], GRW-AS performs targeted exploration of the design, prioritizing paths likely to be relevant for verification. Starting from the target architectural signal's corresponding node, GRW-AS performs multiple, guided random walks through the KG. Each walk is biased by a combination of:

(i) **Local Node Importance**: Prioritizing structurally significant nodes (e.g., modules, key registers), thus guiding the walk toward verification-critical regions of the KG. The importance score is computed as $I(n) = 0.4 \cdot \frac{deg(n)}{max_degree} + 0.6 \cdot T(n)$, where $deg(n)$ is the degree of node n, max_degree is the maximum degree of any node in the KG, and $T(n)$ represents a mapping from a node type to its semantic significance which can be manually adjusted using domain knowledge of the design under verification. In our current implementation, we use a uniform weight for all nodes.

(ii) **Directional Guidance**: Favoring paths that lead towards other known architectural signals. In the initialization phase, we compute shortest paths between all pairs of architectural signals and store the "next hop" for each signal-to-signal path. For a candidate node c during the walk, the direction score is calculated as $D(c) = \frac{\sum_{t \in \text{architectural_signals}} \mathbb{I}(c \text{ is next hop to } t)}{|\text{architectural_signals}|}$, where the indicator function equals 1 if c is the first node along the shortest path from the current node to target architectural signal t. This effectively biases the walk towards nodes that serve as "stepping stones" to reach other architectural signals.

(iii) **Exploration novelty**: Encouraging exploration of less-visited areas of the graph by prioritizing unvisited nodes. Quantified as $N(c) = \begin{cases} 1.0 & \text{if } c \notin visited \\ 0.0 & \text{otherwise} \end{cases}$, where $visited$ is the set of already visited nodes in the current walk, which is dynamically updated during each walk, making the sampling adaptive.

These factors are combined to compute transition probabilities at each step according to the formula: $P(c) = \alpha \cdot I(c) + \beta \cdot D(c) + \gamma \cdot N(c)$ where α, β, and γ are configurable weights controlling the influence of each factor. The probabilities are normalized to sum to 1 before node selection. Walks terminate either upon reaching another architectural signal or exceeding a predefined step budget.

IV. EXPERIMENTS

We evaluate our approach, ASSERTIONFORGE, on five designs. All of the datasets and model implementation will be released upon paper acceptance.

A. Dataset and Knowledge Graph Statistics

We evaluate our approach on five diverse hardware designs. These designs represent a range of complexities and functionalities in hardware systems, from communication protocols to microcontrollers and cryptographic modules. For each design, our focus is on generating SVAs for a specific module within the design. Since Verilog allows modules to reference or import other modules, we consider the total number of RTL files associated with each design, as shown in Table I.

B. Baselines and Implementation Details

For all methods, we use GPT-4o [40] as the LLM backend. KG construction follows the GraphRAG framework [17] with default settings, customizing only the prompt for entity and relation extraction. RTL parsing utilizes 'pyverilog' [55]. For SSR, we adopt a multi-scale hierarchical chunking strategy following [8], [33], with chunk sizes of 50, 100, 200, 800, and 3200 tokens and overlap ratios of 0.2 and 0.4. We retrieve the top 20 ranked chunks per signal-name query using a hybrid similarity model combining TF-IDF [51] and Sentence Transformers [48]. The GRW-AS algorithm runs 70 walks per signal with a budget of 100 steps, using $\alpha = 0.3$, $\beta = 0.5$, and $\gamma = 0.2$. The LLM-based pruner retains a maximum of 50 contexts per type and 100 in total.

To ensure a fair comparison and simulate a realistic verification scenario, we limit each method to a maximum of 3 prompts per architectural signal. Global summaries and pruned contexts from SSR and GRW-AS are grouped by type, and then distributed across these prompts, maximizing context inclusion while respecting the GPT-4o's token limit per prompt.

We compare ASSERTIONFORGE against three baselines: (1) **ASSERTLLM** [18]: An LLM-based approach that generates SVAs directly from natural language specifications without constructing a KG. (2) **ASSERTIONFORGE W/O RTL**: This baseline is similar to our full ASSERTIONFORGE, but it does not perform RTL refinement, relying solely on the initial KG $\mathcal{G}_0$ constructed from the design specification. (3)

TABLE I

OVERVIEW OF THE HARDWARE DESIGNS AND KNOWLEDGE GRAPH STATISTICS USED IN OUR EVALUATION. TOKEN COUNTS ARE OBTAINED USING THE CL100K_BASE TOKENIZER [41]. TIME REFERS TO THE KG CONSTRUCTION TIME IN MINUTES.

Design	Source	Description	Spec Pages	Spec Tokens	RTL Files	RTL Tokens	# Nodes	# Edges	Time
APB	[42]	APB to I2C interface	12	2,634	1	1,116	128	225	4.25
ETHMAC	[42]	Ethernet MAC transmit control	102	33,754	4	6,262	1332	2502	8.10
OPENMSP430	[18]	MSP430 microcontroller	129	34,133	29	116,093	4636	5800	6.67
SOCKIT	[18]	FPGA-based 1-wire master	29	13,102	4	11,547	678	1400	4.41
UART	[18]	UART to Bus interface	10	3,103	6	9,235	1490	1815	1.60

TABLE II

EVALUATION RESULTS. (#SVA: TOTAL ASSERTIONS GENERATED; #SYNC: SYNTACTICALLY CORRECT; #PROVEN: PROVEN OR PASSING; COI STATEMENT/BRANCH/FUNCTIONAL/TOGGLE: COVERAGE METRICS FOR EACH COVERAGE MODEL UNDER COI.)

Model	#SVA	#SynC	#Proven	COI Coverage (%)			
				Statement	Branch	Functional	Toggle
APB							
ASSERTLLM	221	170	56	100.00	86.67	84.79	84.16
ASSERTIONFORGE W/O RTL	188	128	22	100.00	86.67	70.05	68.32
ASSERTIONFORGE ORIG KG	551	412	92	**100.00**	**100.00**	**100.00**	**100.00**
ASSERTIONFORGE	**615**	**549**	**220**	**100.00**	**100.00**	**100.00**	**100.00**
ETHMAC							
ASSERTLLM	520	106	15	45.10	44.83	49.01	50.29
ASSERTIONFORGE W/O RTL	1345	331	42	50.98	51.72	71.21	77.46
ASSERTIONFORGE ORIG KG	1365	428	74	94.12	93.10	77.80	72.83
ASSERTIONFORGE	**1673**	**960**	**208**	**100.00**	**100.00**	**99.12**	**98.84**
OPENMSP430							
ASSERTLLM	409	128	55	94.70	95.05	87.32	85.81
ASSERTIONFORGE W/O RTL	1123	216	106	99.08	98.91	88.94	86.95
ASSERTIONFORGE ORIG KG	1202	323	153	99.54	99.52	89.87	87.96
ASSERTIONFORGE	**1600**	**698**	**327**	**100.00**	**99.76**	**90.09**	**88.16**
SOCKIT							
ASSERTLLM	118	49	13	85.71	82.35	88.54	90.46
ASSERTIONFORGE W/O RTL	393	131	41	**100.00**	**100.00**	99.33	99.08
ASSERTIONFORGE ORIG KG	429	**156**	32	**100.00**	**100.00**	**99.78**	**99.69**
ASSERTIONFORGE	**448**	136	**49**	**100.00**	**100.00**	**99.78**	**99.69**
UART							
ASSERTLLM	249	83	29	74.36	78.26	66.78	63.01
ASSERTIONFORGE W/O RTL	233	93	28	74.36	78.26	70.72	68.49
ASSERTIONFORGE ORIG KG	**279**	**174**	**50**	**84.62**	**84.78**	83.88	83.56
ASSERTIONFORGE	253	132	27	**84.62**	**84.78**	**88.82**	**90.41**

ASSERTIONFORGE ORIG KG: This baseline uses the full ASSERTIONFORGE pipeline but uses the vanilla GraphRAG prompt for KG construction instead of our domain-customized prompt with hardware-specific schema and examples.

C. Evaluation Protocol

We use Cadence JASPER (version 2023.12) to evaluate our generated SystemVerilog Assertions (SVAs). We measure the total number of SVAs generated (**#SVA**), the number of syntactically correct SVAs (**#SynC**), and the number of SVAs proven without counterexamples (**#Proven**). Assuming a golden RTL implementation, this work focuses on generating high-quality SVAs; debugging counterexamples to determine root cause (RTL bug versus assertion error) is left for future work. Coverage is evaluated only for *proven* assertions using Jasper's *cone-of-influence (COI)* coverage [10], [34] models. We report **COI Functional Coverage**, **COI Branch Coverage**, **COI Statement Coverage**, and **COI Toggle Coverage**, measuring coverage of high-level functionality, conditional branches, statements, and signal transitions, respectively, within the assertion's influence. COI coverage is obtained using the `check_cov` command in Jasper.

D. Results and Analysis

Table II presents the evaluation results across five hardware designs, where ASSERTIONFORGE consistently outperforms the baselines in both proven assertions and coverage. The gains are especially notable for more complex designs, achieving near-perfect coverage across multiple metrics. Although ASSERTIONFORGE ORIG KG generally surpasses ASSERTLLM, it still falls short of the full ASSERTIONFORGE pipeline, underscoring the impact of our domain-specific KG construction. These results highlight not just the quantity but the quality of our generated assertions — a critical distinction since plausible but incorrect assertions waste computational resources.

E. Ablation Study

To analyze the contribution of each component in ASSERTIONFORGE, we conduct an ablation study on the UART design as in Table III. Removing the *guided random walk algorithm* (No GRW) significantly reduces the number of

TABLE III
ABLATION STUDY RESULTS ON THE UART DESIGN, SHOWING THE IMPACT OF REMOVING INDIVIDUAL COMPONENTS FROM OUR FRAMEWORK.

Model Variant	#SVA	#SynC	#Proven	COI Coverage (%)			
				Statement	Branch	Functional	Toggle
Full ASSERTIONFORGE (all components)	253	132	27	84.62	84.78	88.82	90.41
No Summaries (w/o global summarization)	236	76	31	84.62	84.78	78.29	75.80
No RAG (w/o retrieval-augmented generation)	311	128	38	84.62	84.78	83.55	83.11
No GRW-AS (w/o guided random walk)	243	115	26	84.62	84.78	83.55	83.11
No Pruner (w/o LLM-based context pruning)	281	144	20	84.62	84.78	83.88	83.56
Spec-RTL Concatenation	91	36	9	74.36	78.26	70.39	68.04

proven assertions, highlighting its importance in exploring relevant design paths. The *global summarization component* (No Summaries) drastically reduces syntactically correct assertions while producing mixed effects on coverage metrics. The baseline approach that simply concatenates specification and RTL text (Spec-RTL Concat) performs worst across all metrics, demonstrating that naively combining all available information overwhelms the LLM and is impractical for industry-scale designs with substantially larger specifications and RTL.

F. Case Study: Knowledge Graph Visualization

Figure 2 visualizes KGs constructed from the OPENMSP430 design specification, contrasting the impact of our domain-specific schema.

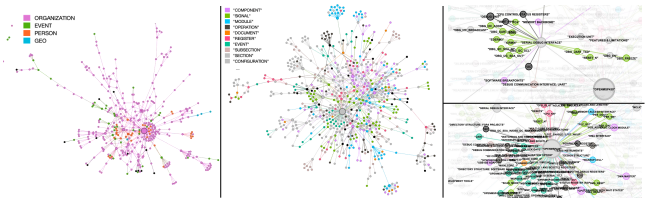

Fig. 2. Visualization of the KGs from OPENMSP430 using [6]. Node colors represent different types of entities, such as modules and signals. Node colors indicate the *type* attribute of each entity. Left: KG generated via the original entity extraction prompt of GraphRAG. Middle: KG generated via our domain-customized prompt. Right: Two zoomed-in views of the KG, highlighting key entities with their labels.

G. Case Study: SVAs Generated for ETHMAC

We illustrate that ASSERTIONFORGE generates high-quality SVAs capturing essential design properties on the ETHMAC design in Figure 3, an Ethernet MAC transmit control module. The first SVA ensures 'ByteCnt' resets and 'TxFlow' halts during reset, which is crucial for maintaining transmission integrity, while the second verifies the correct assertion of 'WillSendControlFrame' during the transmission of control frames, ensuring proper data transmission.

H. Case Study: Paths Extracted by GRW-AS Algorithm

To illustrate the effectiveness of our GRW-AS algorithm, we examine key paths extracted from two designs' KGs that may have contributed to generating high-quality SVAs. Due to space constraints, we show selected subpaths that capture the essential signal relationships.

Path 1: UART Transmission Control. This path reveals critical dependencies:

Selected SVAs Generated for ETHMAC

Plan: Validate that when 'ResetByteCnt' and 'xReset' are active, 'ByteCnt' is set to zero and 'TxFlow' halts within the next clock cycle. **SVA:**

```
@(posedge MTxClk)
  (ResetByteCnt == 1 && TxReset == 1)
  |-> (ByteCnt == 0 && TxFlow == 0);
```

Plan: Ensure that 'WillSendControlFrame' is asserted just before 'TxStartFrmIn' is asserted and remains asserted until 'TxEndFrmIn' is asserted. **SVA:**

```
@(posedge MTxClk)
  ((WillSendControlFrame && TxStartFrmIn)
   && !TxEndFrmIn)
  |-> WillSendControlFrame;
```

Fig. 3. Examples of SVAs generated by ASSERTIONFORGE.

```
tx_busy (port) → uart_tx (module) →
uart_top (module) → new_tx_data (port) →
data_stability (verification_point)
```

Together with global summaries and signal-specific context, ASSERTIONFORGE synthesized this path information to generate this key SVA:

```
@(posedge clock) (!tx_busy && new_tx_data)
    |-> ##1 tx_busy
```

Path 2: APB Interrupt Generation. This subpath exposes interrupt control flow:

```
INT_TX (assignment) → apb (module) →
TX_EMPTY (port) → data_stability (point) →
reset_behavior (point) → PRESETn (port)
```

These KG-derived insights, combined with specification snippets and global design context, led to critical SVAs for FIFO status and reset behavior:

```
@(posedge PCLK) TX_EMPTY |=> INT_TX
@(posedge PCLK) (!PRESETn) |-> TX_EMPTY
```

V. CONCLUSION AND FUTURE WORK

We introduced ASSERTIONFORGE, a novel approach for generating SystemVerilog Assertions (SVAs) that integrates Knowledge Graph (KG) and Register-Transfer Level (RTL) information. Experimental results on five designs demonstrate significant improvements over baseline methods. While our approach shows promising results, future work could explore more sophisticated KG construction techniques, methods to further improve functional coverage, and extensions to other hardware design automation tasks such as automatic code generation and design error detection.

REFERENCES

[1] Bilal Abu-Salih. Domain-specific knowledge graphs: A survey. *Journal of Network and Computer Applications*, 185:103076, 2021.

[2] Mohammad Akyash and Hadi Mardani Kamali. Self-hwdebug: Automation of llm self-instructing for hardware security verification. *arXiv preprint arXiv:2405.12347*, 2024.

[3] Amira A Alshazly, Ahmed M Elfatatry, and Mohamed S Abougabal. Detecting defects in software requirements specification. *Alexandria Engineering Journal*, 53(3):513–527, 2014.

[4] Avinash Ayalasomayajula, Rui Guo, Jingbo Zhou, Sujan Kumar Saha, and Farimah Farahmandi. Lasp: Llm assisted security property generation for soc verification. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, pages 1–7, 2024.

[5] Lars Backstrom and Jure Leskovec. Supervised random walks: predicting and recommending links in social networks. In *Proceedings of the fourth ACM international conference on Web search and data mining*, pages 635–644, 2011.

[6] Mathieu Bastian, Sebastien Heymann, and Mathieu Jacomy. Gephi: An open source software for exploring and manipulating networks, 2009.

[7] Zhen Bi, Jing Chen, Yinuo Jiang, Feiyu Xiong, Wei Guo, Huajun Chen, and Ningyu Zhang. Codekgc: Code language model for generative knowledge graph construction. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 23(3):1–16, 2024.

[8] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.

[9] Antoine Bosselut, Hannah Rashkin, Maarten Sap, Chaitanya Malaviya, Asli Celikyilmaz, and Yejin Choi. Comet: Commonsense transformers for automatic knowledge graph construction. *ACL*, 2019.

[10] Hana Chockler, Orna Kupferman, and Moshe Y Vardi. Coverage metrics for formal verification. In *Correct Hardware Design and Verification Methods: 12th IFIP WG 10.5 Advanced Research Working Conference, CHARME 2003, L'Aquila, Italy, October 21-24, 2003. Proceedings 12*, pages 111–125. Springer, 2003.

[11] Edmund M Clarke, Thomas A Henzinger, Helmut Veith, and Roderick Bloem. *Model checking*. MIT press, 2018.

[12] Colin Cooper, Sang Hyuk Lee, Tomasz Radzik, and Yiannis Siantos. Random walks in recommender systems: exact computation and simulations. In *Proceedings of the 23rd international conference on world wide web*, pages 811–816, 2014.

[13] Luciano da Fontoura Costa and Gonzalo Travieso. Exploring complex networks through random walks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, 75(1):016102, 2007.

[14] Zhuyun Dai and Jamie Callan. Deeper text understanding for IR with contextual neural language modeling. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 985–988, 2019.

[15] Alessandro Danese, Nicolò Dalla Riva, and Graziano Pravadelli. A-team: Automatic template-based assertion miner. In *Proceedings of the 54th Annual Design Automation Conference 2017*, pages 1–6, 2017.

[16] Ashish Darbari. Specifications: The hidden bargain for formal verification. *EDN Network*, September 2022.

[17] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[18] Wenji Fang, Mengming Li, Min Li, Zhiyuan Yan, Shang Liu, Hongce Zhang, and Zhiyao Xie. Assertllm: Generating and evaluating hardware verification assertions from design specifications via multi-llms. *arXiv preprint arXiv:2402.00386*, 2024.

[19] Harry D Foster, Adam C Krolnik, and David J Lacey. Assertion-based verification: Industry myths to realities (invited tutorial). *International Conference on Computer Aided Verification*, pages 5–10, 2008.

[20] Samuele Germiniani and Graziano Pravadelli. Harm: a hint-based assertion miner. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(11):4277–4288, 2022.

[21] Eugene Goldberg. Generation of A complete set of properties. *CoRR*, abs/2004.05853, 2020.

[22] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016.

[23] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

[24] Marti A Hearst. TextTiling: Segmenting text into multi-paragraph subtopic passages. *Computational Linguistics*, 23(1):33–64, 1997.

[25] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and S Yu Philip. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE transactions on neural networks and learning systems*, 33(2):494–514, 2021.

[26] Philip Nicholas Johnson-Laird. *Mental models: Towards a cognitive science of language, inference, and consciousness*. Number 6. Harvard University Press, 1983.

[27] Rahul Kande, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Shailja Thakur, Ramesh Karri, and Jeyavijayan Rajendran. Llm-assisted generation of hardware assertions. *arXiv preprint arXiv:2306.14027*, 2023.

[28] Rahul Kande, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Shailja Thakur, Ramesh Karri, and Jeyavijayan Rajendran. (security) assertions by large language models. *IEEE Transactions on Information Forensics and Security*, 2024.

[29] Minwoo Kang, Mingjie Liu, Ghaith Bany Hamad, Syed Suhaib, and Haoxing Ren. Fveval: Understanding language model capabilities in formal verification of digital hardware. *arXiv preprint arXiv:2410.23299*, 2024.

[30] Amanda Kau, Xuzeng He, Aishwarya Nambissan, Aland Astudillo, Hui Yin, and Amir Aryani. Combining knowledge graphs and large language models. *arXiv preprint arXiv:2407.06564*, 2024.

[31] Aman Kumar and Deepak Narayan Gadde. Generative ai augmented induction-based formal verification. *arXiv preprint arXiv:2407.18965*, 2024.

[32] Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Nv-embed: Improved techniques for training llms as generalist embedding models. *arXiv preprint arXiv:2405.17428*, 2024.

[33] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[34] Ang Li, Hao Chen, JK Yu, Ee Loon Teoh, and Iswerya Prem Anand. A coverage-driven formal methodology for verification sign-off. In *Design and Verification Conference and Exhibition (DVCon) United States*, 2019.

[35] Ke Liang, Lingyuan Meng, Meng Liu, Yue Liu, Wenxuan Tu, Siwei Wang, Sihang Zhou, Xinwang Liu, Fuchun Sun, and Kunlun He. A survey of knowledge graph reasoning on graph types: Static, dynamic, and multi-modal. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[36] Mingjie Liu, Minwoo Kang, Ghaith Bany Hamad, Syed Suhaib, and Haoxing Ren. Domain-adapted llms for vlsi design and verification: A case study on formal verification. In *2024 IEEE 42nd VLSI Test Symposium (VTS)*, pages 1–4. IEEE, 2024.

[37] Karthik Maddala, Bhabesh Mali, and Chandan Karfa. Laag-rv: Llm assisted assertion generation for rtl design verification. In *2024 IEEE 8th International Test Conference India (ITC India)*, pages 1–6. IEEE, 2024.

[38] Bhabesh Mali, Karthik Maddala, Sweeya Reddy, Vatsal Gupta, Chandan Karfa, and Ramesh Karri. Chiraag: Chatgpt informed rapid and automated assertion generation. *arXiv preprint arXiv:2402.00093*, 2024.

[39] Jose L Martinez-Rodriguez, Ivan Lopez-Arevalo, and Ana B Rios-Alvarado. Openie-based approach for knowledge graph construction from text. *Expert Systems with Applications*, 113:339–355, 2018.

[40] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[41] OpenAI. Tokenizer. https://platform.openai.com/tokenizer, 2023. Accessed: 2024-09-18.

[42] OpenCores. Opencores: Open source hardware designs. OpenCores Website, 2024. Accessed: June 28, 2024.

[43] Marcelo Orenes-Vera, Aninda Manocha, David Wentzlaff, and Margaret Martonosi. Autosva: Democratizing formal verification of rtl module

interactions. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pages 535–540. IEEE, 2021.

[44] Marcelo Orenes-Vera, Margaret Martonosi, and David Wentzlaff. Using llms to facilitate formal verification of rtl. *arXiv preprint arXiv:2309.08604*, 2023.

[45] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[46] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*, 2024.

[47] Vaishnavi Pulavarthi, Deeksha Nandal, Soham Dan, and Debjit Pal. Are llms ready for practical adoption for assertion generation?, 2025.

[48] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.

[49] Xuhui Ren, Tong Chen, Quoc Viet Hung Nguyen, Lizhen Cui, Zi Huang, and Hongzhi Yin. Explicit knowledge graph reasoning for conversational recommendation. *ACM Transactions on Intelligent Systems and Technology*, 15(4):1–21, 2024.

[50] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[51] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information processing & management*, 24(5):513–523, 1988.

[52] Kalpesh Sanghvi. From specification to chip: A holistic design approach. *SemiEngineering*, 2014.

[53] Bhaskarjit Sarmah, Benika Hall, Rohan Rao, Sunil Patel, Stefano Pasquali, and Dhagash Mehta. Hybridrag: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. *arXiv preprint arXiv:2408.04948*, 2024.

[54] Cunxi Sun, Christoph Hahn, and Caroline Trippel. Towards improving verification productivity with circuit-aware translation of natural language to systemverilog assertions. In *First International Workshop on Deep Learning-aided Verification (DAV)*, 2023.

[55] Shinya Takamaeda-Yamazaki. Pyverilog: A python-based hardware design processing toolkit for verilog hdl. In *Applied Reconfigurable Computing*, volume 9040 of *Lecture Notes in Computer Science*, pages 451–460. Springer International Publishing, Apr 2015.

[56] Milena Trajanoska, Riste Stojanov, and Dimitar Trajanov. Enhancing knowledge graph construction using large language models. *arXiv preprint arXiv:2305.04676*, 2023.

[57] Shobha Vasudevan, David Sheridan, Sanjay Patel, David Tcheng, Bill Tuohy, and Daniel Johnson. Goldmine: Automatic assertion generation using data mining and static analysis. In *2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*, pages 626–629. IEEE, 2010.

[58] Srikanth Vijayaraghavan and Meyyappan Ramanathan. *A practical guide for SystemVerilog assertions*. Springer Science & Business Media, 2005.

[59] Hong Wang, Jin Liu, Mingyang Duan, Peizhu Gong, Zhongdai Wu, Junxiang Wang, and Bing Han. Cross-modal knowledge guided model for abstractive summarization. *Complex & Intelligent Systems*, 10(1):577–594, 2024.

[60] Yu Wang, Nedim Lipka, Ryan A Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. Knowledge graph prompting for multi-document question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19206–19214, 2024.

[61] George H Weiss and Robert J Rubin. Random walks: theory and selected applications. *Adv. Chem. Phys*, 52:363–505, 1983.

[62] Hongbin Ye, Ningyu Zhang, Hui Chen, and Huajun Chen. Generative knowledge graph construction: A review. *EMNLP*, 2022.

[63] Lingfeng Zhong, Jia Wu, Qian Li, Hao Peng, and Xindong Wu. A comprehensive survey on automatic knowledge graph construction. *ACM Computing Surveys*, 56(4):1–62, 2023.

[64] Yuqi Zhu, Xiaohan Wang, Jing Chen, Shuofei Qiao, Yixin Ou, Yunzhi Yao, Shumin Deng, Huajun Chen, and Ningyu Zhang. Llms for knowledge graph construction and reasoning: Recent capabilities and future opportunities. *World Wide Web*, 27(5):58, 2024.

APPENDIX A
IMPLEMENTATION DETAILS

A. Overall Process

Algorithm 1 outlines the core workflow of ASSERTIONFORGE, integrating specification analysis, RTL parsing, and multi-resolution context synthesis.

Algorithm 1 ASSERTIONFORGE Main Process

Require: Design specification $\mathcal{S}$, RTL code $\mathcal{R}$
Ensure: Generated SVAs $\mathcal{A}$

Parse specification: $\mathcal{S} \leftarrow \text{PDFParser}(\mathcal{S})$
Build initial KG: $\mathcal{G}_0 \leftarrow \mathcal{F}(\mathcal{S})$ {Sec. III-B}
Refine KG with RTL: $\mathcal{G} \leftarrow \psi(\mathcal{G}_0, \mathcal{R})$ {Sec. A-C}
Extract valid architectural signals: $\mathcal{V} \leftarrow \text{RTLSignalExtractor}(\mathcal{R})$

Generate global summary: $\mathcal{C}_g \leftarrow \text{DesignSummarizer}(\mathcal{S}, \mathcal{R}, \mathcal{V})$
for each signal $v_i \in \mathcal{V}$ **do**
  Retrieve signal context: $\mathcal{C}_r \leftarrow \text{SSR}(v_i, \mathcal{S}, \mathcal{R})$
  Extract KG paths: $\mathcal{C}_k \leftarrow \text{GRW-AS}(\mathcal{G}, v_i, \mathcal{V})$ {Sec. III-E}
  Prune contexts: $\mathcal{C}_p \leftarrow \mathcal{P}(\mathcal{C}_g \cup \mathcal{C}_r \cup \mathcal{C}_k)$
  Initialize prompt budget: $\mathcal{B} \leftarrow 3$ {Max prompts per signal}
  for $j \leftarrow 1$ **to** $\mathcal{B}$ **do**
    Construct dynamic prompt: $\mathcal{D}_{i,j} \leftarrow \Omega(\mathcal{C}_p, v_i, j)$ {Sec. A-J}

    Generate NL plans: $\mathcal{P}_{i,j} \leftarrow \mathcal{L}_{\text{plan}}(\mathcal{D}_{i,j})$
    Synthesize SVAs: $A_{i,j} \leftarrow \mathcal{L}_{\text{sva}}(\mathcal{P}_{i,j}, \mathcal{D}_{i,j})$
    Verify assertions: $\text{Jasper}(A_{i,j})$
  end for
end for
Aggregate coverage metrics: $\text{AnalyzeResults}(\bigcup_{i,j} A_{i,j})$

B. Knowledge Graph Construction from Specifications

The creation of a comprehensive Knowledge Graph (KG) from design specifications is a critical first step in our approach. We leverage a customized version of GraphRAG [17] for this purpose, adapting it specifically for hardware verification domain knowledge. The entity extraction process is guided by a detailed prompt that defines the domain-specific schema (Figure 4). This prompt is crucial for ensuring that the LLM extracts relevant entities and relationships from the hardware specifications.

A key advantage of our approach is the adaptability of the knowledge graph schema. Formal verification engineers can modify the entity and relation types based on their domain knowledge and the specific design under verification. For example, when verifying a processor design, additional entity types like "Pipeline Stage," "Execution Unit," or "Branch Predictor" could be added to the schema. Similarly, when verifying a memory controller, types like "Memory Channel," "Timing Parameter," or "Refresh Cycle" might be more relevant.

C. RTL Parsing and Knowledge Graph Refinement

The initial knowledge graph, $\mathcal{G}_0$, constructed from the design specification, is refined by integrating structural and behavioral information extracted from the RTL code, $\mathcal{R}$. This process, denoted as $\mathcal{G} = \psi(\mathcal{G}_0, \mathcal{R})$, is crucial for bridging the

Prompt for Entity Extraction

[The following content is summarized or omitted for brevity: - Goal, Steps, and Additional instructions for hardware design specifications. For the full prompt, please refer to the GraphRAG repository.]
Entity Types:
Design Specification, Section, Subsection, Table, Figure, Author, Module, Submodule, Protocol, Signal, Port, Register, FIFO, Clock, Interrupt, Operation, Frequency, Standard, Reference, Component, Version, Date, Comment, Pin, Configuration, Constraint/Rule, Address, Document, Block
Relation Types:
hasSection, hasSubsection, contains, authoredBy, defines, implements, uses, connectsTo, configures, generatesInterrupt, operatesAt, compliesWith, referencedIn, inputTo, outputFrom, partOf, interfacesWith, triggersOperation, dependsOn, transmitsData, receivesData, has_input, has_output, has_register, performs, uses_protocol, has_constraint, has_rule, describes, references, modifies, creates, closes/fixes, HasSubModule, HasSignal, HasPort, UsesProtocol, UsesClock, UsesAddress, DescribesOperation, TriggersInterrupt, RevisionHistory, belongsToSection, illustrates, hasAuthor, has_port, connected_to, described_in
Examples:
[Example outputs and entity-relationship formats demonstrated in multiple contexts, including hardware design specifications and narrative texts.]
-Real Data-
Entity_types: {entity_types}
Text: {input_text}

Fig. 4. Example prompt for entity extraction (abbreviated). For the full prompt, see: `https://github.com/microsoft/graphrag/blob/main/graphrag/index/graph/extractors/graph/prompts.py`

gap between high-level design intent and low-level implementation details. We achieve this using a custom RTL analysis pipeline built upon the PyVerilog library [55]. Algorithm 2 provides an overview of the RTL parsing procedure.

The RTL parsing and knowledge graph refinement were implemented using Python 3.10.14 and PyVerilog version 1.3.0.

Algorithm 2 RTL Parsing and KG Refinement

Require: Initial KG $\mathcal{G}_0$, RTL code $\mathcal{R}$
Ensure: Refined KG $\mathcal{G}$
$\mathcal{G} \leftarrow \mathcal{G}_0$ {Initialize with specification KG}
$\mathcal{R} \leftarrow$ preprocess_includes($\mathcal{R}$) {Inline included files}
rtl_elements $\leftarrow$ parse_rtl($\mathcal{R}$) {Extract modules, ports, signals, etc.}
dataflow $\leftarrow$ analyze_dataflow($\mathcal{R}$) {Extract signal dependencies}
for each module $m \in$ rtl_elements.modules **do**
  Add module node to $\mathcal{G}$
  Add nodes for module ports, signals, instances, FSMs
  Add edges for structural relationships (containment, connections)
  Add nodes and edges for behavioral relationships (assignments, control flow)
end for
Link specification nodes in $\mathcal{G}_0$ to corresponding RTL nodes in $\mathcal{G}$
Add root node connecting all components if needed for graph connectivity
return $\mathcal{G}$

The parsing process begins by preprocessing the RTL files to handle include directives, inlining the contents of included files to create a self-contained representation. We extract key structural and behavioral information through several specialized methods.

For structural information, we leverage PyVerilog's parser to extract hierarchical component information. Module definitions are identified by traversing the Abstract Syntax Tree (AST) and locating "ModuleDef" nodes. For each module, we extract port declarations ("Input", "Output", "Inout" nodes) from the AST, capturing direction (input/output/inout) and width details (using the "Width" node in the AST). This process determines whether a port is an input or output and extracts its bit width through direct AST analysis. Module instantiations are identified within the parent module's definition via "Instance" nodes, along with their port connections by examining the "portlist" attribute.

For behavioral information, we leverage a multi-strategy approach to identify FSMs. The primary method examines always blocks with clock sensitivity lists that contain case statements, a common FSM implementation pattern. We supplement this with pattern matching to identify state variables, looking for signals named with common patterns like "state", "st_", or "current". For instance, when we encounter code patterns like "always @(posedge clk)" followed by "case(current_state)", we recognize a potential FSM. Within these "always" blocks, we look for "case" statements (represented by "Case" nodes in the AST). We supplement this AST-based approach with pattern matching on signal and parameter names, looking for common FSM-related identifiers like "state", "current_state", "next_state", "st_", or "fsm". This helps identify FSMs even when coding styles deviate from the strict "always @(posedge clk)" and "case(state)" pattern.

Control flow structures are another critical aspect of RTL behavior. We extract conditional statements (if/else), case statements, and loops using a combination of AST traversal and pattern matching. For example, a regex pattern identifies case statement conditions, which often represent state-dependent behavior. Similarly, we analyze assignments, distinguishing between continuous assignments (assign statements) and procedural assignments in always blocks.

Signal assignments are extracted from both "Assign" nodes (for continuous assignments) and from within "always" blocks (for procedural assignments). For each assignment, the left-hand side (LHS) signal (the target of the assignment) and the right-hand side (RHS) expression are recorded. The RHS expression is further analyzed to identify the signals involved, establishing dependencies. We distinguish between blocking and non-blocking assignments, which is crucial for correctly modeling Verilog semantics.

PyVerilog's "VerilogDataflowAnalyzer" is used to construct a dataflow graph. The analyzer's "getBindings" method is used to obtain the dataflow bindings for each signal, revealing its dependencies. We add this dataflow information directly to our knowledge graph by creating edges between signal nodes.

After extracting this RTL information, we refine the knowl-

edge graph as shown in Algorithm 2. New nodes are added for RTL elements, including modules, ports, signals, module instances, FSMs, control flow structures, and assignments. Existing nodes from the initial specification KG ($\mathcal{G}_0$) are preserved. Edges are created to represent:

(i) **Containment:** Modules contain ports, signals, instances, FSMs, and control structures.
(ii) **Instantiation:** A module instantiates another module.
(iii) **Port Connections:** Signals are connected to ports in module instances.
(iv) **Dataflow:** Signals drive other signals through assignments.
(v) **Control Flow:** Control structures (if, case, loops) influence signals/assignments.
(vi) **FSM Structure:** An edge goes from FSM to module.
(vii) **Assignment:** An edge goes from assignment to lhs and rhs.

Finally, a linking process, detailed in Section A-D, establishes connections between nodes in the specification KG ($\mathcal{G}_0$) and the RTL-derived KG ($\mathcal{G}$). The result is a unified KG representing both design intent and implementation details.

D. Fuzzy Signal Name Matching

A critical aspect of bridging the specification and RTL domains is establishing correspondence between entity names that may differ due to naming conventions or abbreviations. Our approach leverages a combination of exact and fuzzy matching techniques to link entities between the specification and RTL knowledge graphs.

For exact matching, we identify direct name references in the specification text using regular expressions, seeking word boundary-anchored occurrences of signal and module names from the RTL. However, hardware specifications often refer to signals using variants of their RTL names. For example, a specification might mention "reset signal" while the RTL uses "rst_n" or "PRESETn".

To address this, we implement a specialized fuzzy matching algorithm that considers:

(i) **Common hardware abbreviations:** We maintain a dictionary mapping between full terms and common hardware abbreviations (e.g., "reset" $\rightarrow$ "rst", "clock" $\rightarrow$ "clk").
(ii) **Case and separator variations:** We normalize names by converting to lowercase and removing separators like underscores, allowing matches between variants like "data_valid" and "DataValid".
(iii) **Active-low signal conventions:** We detect common patterns for active-low signals (e.g., "_n", "_b" suffixes or negation prefixes like "n" or "not_").
(iv) **Levenshtein distance:** For remaining candidates, we calculate edit distance with thresholds proportional to name length, capturing minor typographical variations.

The matching process uses a scoring system where exact matches receive the highest score (1.0), followed by abbreviation expansions (0.9), case/separator variations (0.8), and edit-distance-based matches with scores decreasing as distance increases. We require a minimum match score of 0.6 and prioritize the highest-scoring match when multiple candidates exist.

For example, given an RTL signal "tx_data_valid", our algorithm would successfully match specification mentions of "transmit data valid", "TX_DATA_VALID", or "tx data valid signal". This fuzzy matching is particularly valuable for signals with domain-specific naming, where conventional string similarity metrics alone would fail to establish correct correspondence.

E. RTL Signal Extractor

In the ASSERTIONFORGE framework, the RTL Signal Extractor is essential for identifying and extracting valid signals from the RTL code. This process begins by parsing the RTL code to locate module declarations and their interfaces, typically encompassing `input`, `output`, and `inout` signals. Using regular expressions, the extractor identifies signal declarations, capturing details such as signal direction (`input`, `output`, `inout`), data type (`reg`, `wire`), and bit-width specifications. For instance, a signal declared as `input [7:0] data_in` indicates an 8-bit wide input signal named `data_in`. Once extracted, these signal names are compiled into a set of valid signals, serving as a reference for subsequent stages in the workflow as in Algorithm 1.

F. Global Summarization

The Design Summarizer (Global Summarization) component, denoted as $\mathcal{C}_g \leftarrow \text{DesignSummarizer}(\mathcal{S}, \mathcal{R}, \mathcal{V})$ in Algorithm 1, plays a pivotal role in distilling complex hardware design specifications and RTL code into concise, verification-focused summaries. This step is essential for providing the language model with a comprehensive understanding of the design context prior to assertion generation. Our implementation leverages a multi-faceted summarization approach that targets different aspects of the design through specialized prompts.

The Design Summarizer generates several complementary views of the design. First, it creates a high-level design specification summary (Figure 5) that captures the main functionality and architectural components in 3-5 sentences. This is complemented by an RTL architecture summary (Figure 6) that focuses on module hierarchy and interfaces. Additionally, it produces a comprehensive signals summary (Figure 7) that provides detailed information about each valid signal, including type, bit width, functionality, and interactions with other signals. Finally, it identifies key design patterns and protocols (Figure 8) that have verification implications.

Beyond these global summaries, the Design Summarizer also generates signal-specific descriptions for each target signal during the assertion generation process. This focused analysis, shown in Figure 9, provides detailed context about the specific signal for which assertions are being generated.

The Design Summarizer implements an efficient caching mechanism to avoid redundant LLM calls. The global design summary is generated once and reused across all assertion

Prompt for Design Specification Summary

You are an expert hardware design engineer. Please provide a concise summary (3-5 sentences) of the following hardware design specification. Focus on the main functionality, key components, and architecture. The summary should give a clear high-level understanding of what this design does.
Design Specification: {spec_text}
Provide only the summary, with no additional commentary or introduction.

Fig. 5. Prompt for generating a high-level design specification summary. The prompt emphasizes conciseness while focusing on functionality, key components, and architecture.

Prompt for RTL Architecture Summary

You are an expert hardware design engineer. Please provide a concise summary (3-5 sentences) of the following RTL code. Focus on the module hierarchy, interfaces, and key architectural features.
RTL Code: {rtl_text}
Provide only the RTL architecture summary, with no additional commentary or introduction.

Fig. 6. Prompt for generating an RTL architecture summary, focusing on module hierarchy and interfaces.

Prompt for Comprehensive Signals Summary

You are an expert hardware verification engineer. Please analyze the following design specification and RTL code to provide a comprehensive summary of the signals in the design. For each signal, include details about:
Signal name
Signal type (input, output, inout, internal, clock, reset, etc.)
Bit width (e.g., 1-bit, 8-bit, 32-bit)
Functionality and purpose
Key interactions with other signals
Valid Signals: signals_str
Design Specification: {spec_text}
RTL Code: {rtl_text}
Focus on the signals listed above. If the RTL/spec doesn't provide information for a signal, make your best inference. Format your response as a list with each signal having its own paragraph that includes all the details mentioned above. Be concise yet complete.

Fig. 7. Prompt for generating a comprehensive summary of all signals in the design, including detailed technical characteristics.

Prompt for Design Patterns Summary

You are an expert hardware design engineer. Please analyze the following design specification and RTL code to identify and summarize key design patterns, protocols, or verification-critical structures. Examples might include handshaking protocols, state machines, pipelines, arbiters, or clock domain crossings.
Design Specification: {spec_text}
RTL Code: {rtl_text}
Provide a concise summary (5-10 sentences) of the key design patterns and their verification implications.

Fig. 8. Prompt for identifying and summarizing key design patterns and protocols with verification implications.

Prompt for Signal-Specific Description

You are an expert hardware verification engineer. Please provide a detailed description of the signal '{signal_name}' based on the following specification and RTL.
Design Specification: {spec_text}
RTL Code: {rtl_text}
Include in your description:
The precise function of this signal Its type (input, output, inout, internal, etc.) and bit width Its timing characteristics (synchronous/asynchronous, edge-triggered, etc.) Key relationships with other signals How it affects or is affected by the overall system behavior Any special conditions or corner cases related to this signal Write 3-5 sentences with comprehensive, verification-focused details.

Fig. 9. Prompt for generating a detailed description of a specific signal, focusing on its function, characteristics, and interactions.

generation tasks, while signal-specific descriptions are cached individually. This approach significantly reduces the overall LLM query cost while maintaining comprehensive context for assertion generation.

When constructing the final context for assertion generation, the summarizer combines these various summary components in a hierarchical manner, starting with the global design overview, followed by the RTL architecture summary, the target signal description, the comprehensive signals summary, and finally the design patterns summary. This layered approach provides the assertion generation model with both broad design context and signal-specific details, enabling it to generate more accurate and relevant SystemVerilog assertions.

G. Signal-Specific Retrieval (SSR)

The Signal-Specific Retrieval (SSR) component, represented as $\mathcal{C}_r \leftarrow \text{SSR}(v_i, \mathcal{S}, \mathcal{R})$ in Algorithm 1, is responsible for retrieving relevant contextual information from both specification documents and RTL code based on signal-specific queries.

Our SSR implementation serves as a critical middle-resolution layer in our multi-level context synthesis framework, bridging the gap between the high-level global summarization and the fine-grained graph traversal of GRW-AS. While global summarization captures design-wide architectural intent and GRW-AS explores detailed signal-to-signal relationships, SSR provides essential functional context at the module and subsystem level. This intermediate resolution is crucial for hardware verification, as many behavioral specifications and protocol sequences exist precisely at this middle level of abstraction.

To effectively capture this intermediate information, we adopt a multi-scale hierarchical chunking strategy inspired by the work of Lewis et al. [33] and Borgeaud et al. [8], dividing documents into chunks of varying granularities that form what Robertson and Callan [50] refer to as a "resolution pyramid." At the finest resolution (50 tokens), the system captures precise signal definitions and immediate interactions. The middle resolutions (100-200 tokens) capture functional behaviors and protocol sequences that typically span multiple sentences or paragraphs. The coarsest resolutions (800-3200

tokens) preserve architectural relationships and system-level behaviors that might be missed in smaller chunks.

To ensure coherent context across chunk boundaries, we implement overlapping chunks with ratios of 0.2 and 0.4, following the approach proposed by Dai and Callan [14]. This overlapping strategy addresses the "boundary fragmentation problem" identified by Hearst [24], where critical information can be split across adjacent chunks. By creating strategic overlaps, we ensure that semantically coherent units (like descriptions of protocol sequences that might span multiple paragraphs) remain intact in at least some retrieved chunks.

Our retrieval framework uses two complementary methods for finding relevant context. First, we use TF-IDF to convert text chunks into sparse vectors that emphasize rare, distinguishing terms. Second, we use Sentence Transformers to generate dense semantic vectors that capture contextual meaning. For retrieval, we use the signal name (e.g., "tx_valid" or "data_ready") as the query term. We compute cosine similarity between this query and each chunk using both vector representations independently, then combine the scores with an average. This dual approach balances exact signal name matching with semantic relevance, allowing us to retrieve context even when signal names are referenced indirectly.

To further refine the retrieval process, future enhancements could involve developing specialized embedding models fine-tuned on hardware design corpora. Such models would be adept at capturing the unique linguistic patterns and terminologies prevalent in hardware design documents, thereby improving retrieval accuracy. For instance, NV-Embed [32], a generalist embedding model, has demonstrated significant improvements in embedding tasks by incorporating architectural innovations and specialized training procedures. Adapting similar techniques to create embeddings tailored for hardware design verification could substantially enhance the performance of the SSR component.

H. Guided Random Walk with Adaptive Sampling (GRW-AS)

GRW-AS discovers semantically relevant paths through the knowledge graph using biased random walks guided by verification priorities. Algorithm 3 outlines the core procedure, implementing the three key biases defined in Section III-E.

Algorithm 3 GRW-AS Path Discovery

Require: Knowledge graph $\mathcal{G}$, Start node v_s, signals $\mathcal{V}$
Ensure: Natural language path description L
$P \leftarrow [v_s]$, $v \leftarrow v_s$ {Initialize path}
while step $\leq$ budget **and** $\mathcal{V} \neq \emptyset$ **do**
  $\mathcal{N} \leftarrow \{u | (v, u) \in \mathcal{E}\}$ {Neighbor nodes}
  $\mathbf{P} \leftarrow [\alpha I(u) + \beta D(u) + \gamma N(u)\ \forall u \in \mathcal{N}]$
  $v' \leftarrow$ sample$(\mathcal{N}, \mathbf{P})$ {Biased selection}
  P.append(v'), $v \leftarrow v'$
  if $v' \in \mathcal{V}$ **then**
    $\mathcal{V} \leftarrow \mathcal{V} \setminus \{v'\}$
  end if
end while
$L \leftarrow$ ConvertToNL(P) {Natural Language (NL) description}
return L

An example of a generated path description is shown in Figure 10. This example shows a path starting from the "baud_limit" port in the UART module, traversing through transmit and receive logic, and discovering critical signals through diverse relationship types. The description includes the type of each node (e.g., "port", "module", "verification_point", "protocol_pattern") and the module it belongs to, along with the varied relationships between consecutive nodes (e.g., "drives", "input to", "involves", "includes"). While our current work focuses on representing paths as human-readable text, which we hypothesize is more natural for the LLM, future work could explore extracting subgraphs or motifs from the knowledge graph.

I. LLM-based Context Pruner

The pruner analyzes each context's relevance, information density, and complementarity with other selected contexts, considering factors like explicit mentions of the target signal, descriptions of related signals, protocol specifications, and timing/behavioral constraints. As shown in Figure 11, the LLM is prompted to act as an expert verification engineer, receiving the original query, target signal name, and a set of contexts with metadata (source type, retrieval score, etc.). Crucially, the prompt instructs the LLM to select between a minimum and maximum number of contexts of the given type, explicitly encouraging the inclusion of even partially relevant information. The LLM outputs a list of indices indicating the selected contexts (e.g., "Selected contexts: [0, 2, 5]"). Our implementation defaults to a maximum of 50 contexts per type and 100 total contexts, with a minimum of 2 contexts per type enforced whenever possible, promoting diversity.

J. Multi-Resolution Context Synthesis

After pruning the contexts from various sources, the next critical step is generating effective verification plans and assertions through Multi-Resolution Context Synthesis. This process, denoted as $\mathcal{D}_{i,j} \leftarrow \Omega(\mathcal{C}_p, v_i, j)$ in Algorithm 1, dynamically constructs prompts that harmonize different context types across multiple resolution levels.

Our implementation assembles relevant contexts from SSR (middle resolution) and GRW-AS (fine-grained resolution), along with global design summaries (coarse resolution). For each signal, the system generates up to three distinct prompts ($\mathcal{B} = 3$ in our implementation), each containing complementary context combinations to maximize coverage of verification scenarios. These prompts respect a maximum token limit of 75% of the LLM's context window, reserving space for the model's response.

The two-stage verification process begins with natural language (NL) plan generation, followed by SystemVerilog Assertion (SVA) synthesis. Figure 12 illustrates the prompt structure used for NL plan generation.

After generating NL plans, the system synthesizes formal SVAs as shown in Figure 13. This stage uses the same context synthesis approach but adds signal-specific context and in-context learning examples to guide correct assertion syntax.

```
GUIDED RANDOM WALK FROM baud_limit (port)
Located in module: baud_gen
Path length: 57 nodes, discovered signals:
tx_busy, ce_16, new_rx_data, rx_data,
    new_tx_data, baud_clk, baud_freq, clock

Signal flow path:
  baud_limit (port in baud_gen) drives
      baud_limit (port in uart_top)
  baud_limit (port in uart_top) input to
      uart_top (module)
  uart_top (module) part of
      new_tx_data_assignment (assignment in
      uart_top)
  new_tx_data_assignment (assignment in
      uart_top) assigns to new_tx_data (port
      in uart_top)
  new_tx_data (port in uart_top) involves
      data_stability (verification_point in
      uart_top)
  data_stability (verification_point in
      uart_top) involves baud_clk (port in
      uart_top)
  baud_clk (port in uart_top) includes
      rtl_111 (protocol_pattern in uart_top)
  rtl_111 (protocol_pattern in uart_top)
      found in uart_top (module)
  ...
  data_buf_assignment (assignment in
      uart_top) used in tx_data (port in
      uart_top)
  tx_data (port in uart_top) drives tx_data
      (port in uart_tx)
  tx_data (port in uart_tx) involves
      data_stability (verification_point in
      uart_tx)
  data_stability (verification_point in
      uart_tx) involves clock (port in
      uart_tx)
  clock (port in uart_tx) drives clock (port
      in uart_rx)
  ...
  rx_data (port in uart_rx) includes rtl_108
      (protocol_pattern in uart_rx)
  rtl_108 (protocol_pattern in uart_rx)
      includes new_rx_data (port in uart_rx)
  ... [truncated]
```

Fig. 10. Example of a path description generated by GRW-AS.

Prompt for LLM Context Pruner

You are an expert hardware verification engineer evaluating contexts to be used in generating verification plans for signal '{signal_name}'.
QUERY: {query}
YOUR TASK: Select between {min_selection} and {max_selection} contexts of type '{context_type}' that could help with verification.
IMPORTANT NOTES:
- Select at least {min_selection} contexts even if they seem only indirectly relevant
- Consider both explicit mentions of '{signal_name}' and general system information
- Partial information about interfaces, protocols, and system behavior is still valuable
- When in doubt, include rather than exclude contexts
CONTEXTS TO EVALUATE:
[CONTEXT 0] {context_0}
—-
[CONTEXT 1] {context_1}
—-
...
SELECTION INSTRUCTIONS:
1. SELECT AT LEAST {min_selection} CONTEXTS, even if only partially relevant
2. Focus on contexts that might help verify {signal_name}
3. Output your selection using ONLY the format "Selected contexts: [list of indices]"
For hardware verification, we need information about:
- Signal connections and dependencies
- Timing requirements
- Protocol details
- State transitions
- Interfaces

Fig. 11. Prompt template for the LLM-based Context Pruner, showing how contexts from different sources are presented for evaluation.

Prompt for NL Test Plan Generation

Generate natural language test plans for signal '{signal_name}'.
Relevant Context: {global_summary}
RAG Context: {rag_context}
GRW-AS Context: {grw_context}
CRITICAL - Valid Signal Names (USE ONLY THESE SIGNALS): {valid_signals}
Examples:
{examples}
Generate diverse test plans based on the given information. Each test plan should be on a new line and start with 'Plan: '.

Fig. 12. Prompt template for natural language verification plan generation, showing how multi-resolution contexts are integrated to guide the generation of signal-specific test plans.

APPENDIX B
EFFICIENCY ANALYSIS

We analyze ASSERTIONFORGE's computational efficiency across the four designs. Table IV summarizes the time distribution across major workflow stages, demonstrating our framework's practical scalability.

Our analysis reveals that ASSERTIONFORGE introduces reasonable computational overhead while enabling comprehensive verification. While the initial Knowledge Graph construction from specifications varies across designs (2.72-10.65%), the subsequent KG refinement from RTL is remarkably efficient at less than 0.4% of total runtime for all designs.

The LLM-based components show good scalability. The dominant computational factor remains Jasper formal verification (57-78% of runtime), which is an inherent cost in any verification approach. This verification phase could be parallelized in future implementations to further reduce overall time, though our current implementation evaluates each assertion sequentially.

It's worth noting that actual running times may vary based on factors such as the LLM backend engine speed, server

TABLE IV
TIME DISTRIBUTION ACROSS VERIFICATION STAGES FOR DIFFERENT HARDWARE DESIGNS

Component	APB	ETHMAC	OPENMSP430	SOCKIT	UART
KG Construction from Spec	4.08%	10.65%	2.72%	0.62%	3.15%
KG Refinement from RTL	0.01%	0.07%	0.33%	0.01%	0.07%
Context Summarization	1.34%	0.00%	3.35%	1.17%	3.65%
NL Plan Generation	8.73%	5.12%	9.97%	3.97%	14.41%
SVA Synthesis	12.14%	5.83%	14.03%	4.48%	20.42%
Jasper Verification	73.26%	77.82%	69.31%	90.34%	57.00%
Other Operations	0.44%	0.51%	0.29%	0.41%	1.30%
Total Runtime	1h 47m	1h 25m	4h 12m	4h 14m	52m 20s

Prompt for SVA Generation

Generate SystemVerilog Assertions based on the following information:
Relevant Context: {global_summary}
{signal_specific_summary}
RAG Context: {rag_context}
GRW-AS Context: {grw_context}
Natural Language Test Plans for signal '{signal_name}':
Plan 1: {plan_1}
Plan 2: {plan_2}
Plan 3: {plan_3}
Examples:
{examples}
Generate one SVA for each of the provided natural language test plans.
Enclose each SVA in triple backticks (“‘) and prefix it with 'SVA:'.

Fig. 13. Prompt template for SVA generation, including multi-resolution contexts, signal-specific natural language plans, and examples demonstrating the expected SVA format.

load, and hardware configuration used for verification. The figures presented here represent typical performance observed in our experimental setup and are intended to illustrate the relative distribution of computational effort rather than absolute benchmarks. In practice, users may observe faster or slower performance depending on their specific environment and the optimizations applied to each component.

APPENDIX C
MORE DETAILS ON EXPERIMENTS

A. Cone-of-Influence Coverage Metrics

When measuring verification quality, we need to look beyond just counting assertions to examine how well they actually test the design's behavior. We chose Cadence JASPER's cone-of-influence (COI) coverage metrics as our quality indicator because they offer deeper insights into each assertion's effectiveness [10].

We use four complementary COI metrics to capture different verification aspects. **COI Statement Coverage** shows what percentage of RTL statements within the assertion's influence are tested, giving us basic code coverage. **COI Branch Coverage** reveals how well the assertion tests conditional paths in the logic, which is essential for verifying how the design makes decisions. **COI Functional Coverage** gauges whether the assertion truly verifies high-level requirements rather than just checking implementation details.

COI Toggle Coverage adds another dimension by tracking signal transitions within the assertion's reach, helping us find static logic that other metrics might overlook. These four metrics together give us a balanced view of assertion quality since each looks at different verification aspects. For example, an assertion might hit most statements but miss important branching logic, or it might check branches without exercising key signals.

B. Impact of Prompt Count Parameter

We conducted a parameter sensitivity study on the maximum prompt count parameter $\mathcal{B}$, which controls how many distinct prompts are generated per architectural signal. Table V shows the results of varying $\mathcal{B}$ from our baseline value of 3 to higher values of 6 and 9 for the UART design.

Our results demonstrate that increasing the maximum prompt count $\mathcal{B}$ generally leads to more proven assertions, with the total count rising from 27 at $\mathcal{B} = 3$ to 61 at $\mathcal{B} = 9$. This improvement is expected, as each additional prompt contains complementary context combinations that can capture different verification scenarios. However, we observe that the relationship is not linear, with diminishing returns as $\mathcal{B}$ increases.

Interestingly, we observe that coverage metrics do not consistently improve with increasing $\mathcal{B}$. For example, COI Functional Coverage actually decreases slightly from 88.82% at $\mathcal{B} = 3$ to 83.93% at $\mathcal{B} = 6$ before recovering to 89.14% at $\mathcal{B} = 9$. This non-monotonic behavior suggests that while more prompts generally lead to more assertions, the quality and diversity of those assertions depend on complex interactions between the contexts selected for each prompt.

In practice, the choice of $\mathcal{B}$ represents a trade-off between verification thoroughness and computational resources. Larger values yield more comprehensive verification but incur higher computational costs for both LLM inference and assertion verification. Based on our experiments, $\mathcal{B} = 3$ offers a reasonable balance for practical applications, while higher values may be justified for critical components requiring exhaustive verification.

C. Testbench SVAs Generated by LLM on the OPENMSP430 Design

Figure 14 demonstrates SVAs generated by ASSERTIONFORGE for the OPENMSP430 microcontroller, spanning clock

TABLE V
EFFECT OF VARYING MAXIMUM PROMPT COUNT PARAMETER $\mathcal{B}$ ON THE UART DESIGN.

$\mathcal{B}$ Value	#SVA	#SynC	#Proven	COI Coverage (%)			
				Statement	Branch	Functional	Toggle
$\mathcal{B} = 3$ (default)	253	132	27	84.62	84.78	88.82	90.41
$\mathcal{B} = 6$	394	255	53	84.62	84.78	83.93	83.64
$\mathcal{B} = 9$	397	197	61	84.62	84.78	89.14	90.87

domains, DMA behavior, interrupt handling, and debug interfaces. These automatically synthesized assertions exhibit three key characteristics: temporal correctness (sequencing requirements), protocol compliance (interface adherence), and data integrity (preventing unknown states).

These SVAs exhibit several features including multi-clock checks (A0), protocol timing requirements (A49), error propagation (A110), and cross-domain triggers (A23). This comprehensive coverage stems from the KG's unified representation of both specification requirements and RTL implementation details.

The guided random walks (GRW-AS) algorithm was particularly effective at discovering relationships between seemingly unrelated signals like "cpu_en" and "dco_wkup" by traversing the "openMSP430.v" connectivity paths. Additionally, examining the RTL implementation revealed critical behaviors not explicitly documented in the specification, such as the timing relationship between "reset_n" and "puc_rst" (A283) and the exact power management protocol between "dco_enable" and "dco_wkup" (A23).

Despite the improvements in assertion quality and coverage achieved by ASSERTIONFORGE, Table II shows coverage is still not 100% complete. Future work includes leveraging existing assertion libraries and learning from well-verified designs via RAG or in-context learning techniques. Additionally, iterative KG refinement, hardware-specific LLM fine-tuning, and implementing verification feedback loops could further enhance coverage.

Selected SVAs for OPENMSP430 Generated by ASSERTIONFORGE

Clock Domain Validation:

```
// A0: ACLK/SMCLK synchronization
@property p0
@(posedge smclk or aclk)
((smclk_en && !aclk_en) |-> (aclk == smclk));
```

Reset Behavior:

```
// A22: Reset propagation
@property p22
@(posedge mclk) (puc_rst) |-> ##1 !aclk_en;
```

DMA Transaction Safety:

```
// A109: Valid DMA input data
@property p109
@(posedge mclk) (dma_en && dma_we)
|-> (dma_din !== 'hX);
```

Interrupt Handling:

```
// A180: IRQ-ACK consistency
@property p180
@(posedge mclk) (irq == 0) |-> irq_acc == 0;
```

Clock Gating:

```
// A307: SMCLK stability
@property p307
@(posedge mclk) (smclk_en && cpu_en)
|-> $isunknown(smclk) == 0;
```

Memory Access Validation:

```
// A110: DMA address bounds
@property p110
@(posedge mclk) (dma_en &&
(dma_addr > 'hFFFF)) |-> (dma_resp == 1'b1);
```

Debug Interface Protocol:

```
// A49: I²C SDA hold time
@property p49
@(posedge dbg_i2c_scl)
(dbg_en && dbg_i2c_sda_in |-> dbg_i2c_sda_out);
```

Power Management:

```
// A23: DCO wakeup trigger
@property p23
@(posedge mclk)
(!dco_enable && cpu_en) |-> dco_wkup;
```

Asynchronous Event Handling:

```
// A231: NMI to handler mapping
@property p231
@(posedge mclk)
disable iff (puc_rst) $rose(nmi)
|-> $rose(cpu.NMI_handler);
```

Reset Assertion:

```
// A283: Reset pin behavior
@property p283
@(posedge mclk)
(reset_n == 1'b0) |-> (puc_rst == 1'b1);
```

Fig. 14. Representative SVAs for OPENMSP430 demonstrating ASSERTIONFORGE's ability to capture diverse verification concerns..